LLM-as-a-Judge Reliability

Phase 1 Combined Presentation

Parsa Gholami, Arya AlYasin, Mohsen Shirazi

Sharif University of Technology

Table of Contents

Paper 4:
Large Language Models are not Fair Evaluators

Table of Contents – Paper 4

Introduction & The Problem

The Evaluation Paradigm

- **The Shift:** As LLMs advance, evaluating their alignment with human intent is critical.
- **The Solution:** Researchers increasingly use LLMs like GPT-4 as automated referees to score and compare candidate models due to high human annotation costs.
- **The Unknown:** It remains unclear how reliable LLMs actually are as evaluators against perturbations.

The Core Discovery: Positional Bias

The Achilles' Heel of LLM Evaluators

- **Positional Bias:** The LLM-as-evaluator paradigm has a severe flaw. Quality ranking can be hacked by altering the order of candidates.
- **Model Preferences:** GPT-4 consistently prefers the *first* displayed candidate, while ChatGPT strongly favors the *second*.
- **The Flip:** Merely swapping the presentation order can completely reverse the evaluation outcomes.

Visualizing the Bias

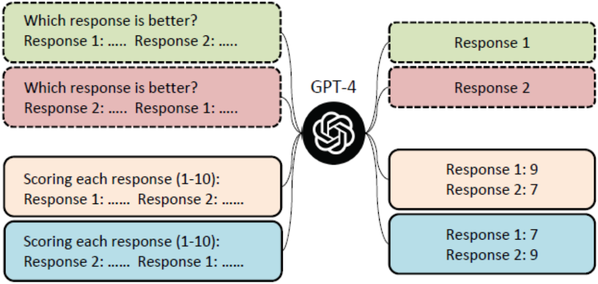

Figure 1: Simply changing the order of candidate responses leads to overturned comparison results, even though we add the command "ensuring that the order in which the responses were presented does not affect your judgment" into the prompt.

Figure 1: Swapping order overturns comparison results.

Quantifying the Unfairness

- **Conflict Rate:** A metric introduced to measure the proportion of conflicting results given by the evaluator when candidate positions are swapped.
- **The Formula:**

$$\text{Conflict Rate} = \frac{\text{Number of Conflicting Evaluations}}{\text{Total Number of Evaluations}}$$

- **Extreme Volatility:** ChatGPT exhibits a massive conflict rate of up to 82.5% when comparing Vicuna-13B to ChatGPT.

Win Rate Fluctuation Data

Evaluators	Vicuna-13B v.s. Other Models	Vicuna-13B Win Rate		Conflict Rate
		as Assistant1	as Assistant2	
GPT-4	Vicuna-13B v.s. ChatGPT	51.3%	23.8%	37 / 80 (46.3%)
GPT-4	Vicuna-13B v.s. Alpaca-13B	92.5%	92.5%	4 / 80 (5.0%)
ChatGPT	Vicuna-13B v.s. ChatGPT	2.5%	82.5%	66 / 80 (82.5%)
ChatGPT	Vicuna-13B v.s. Alpaca-13B	37.5%	90%	42 / 80 (52.5%)

Table 2: The Win Rate of Vicuna-13B significantly fluctuates when positioned as Assistant 1 and Assistant 2, with GPT-4 and ChatGPT serving as evaluators. Conflict Rate refers to the proportion of conflicting results given by the same evaluator when simply changing the position of two models.

Figure 2: Win rate fluctuation and Conflict Rates across different models.

When Does Bias Hit the Hardest?

- The degree of positional bias correlates directly with the quality difference between the two responses.
- **Small Gap:** When candidate quality is very similar, the LLM is significantly more likely to produce conflicting results based purely on position.
- **Large Gap:** If one response is obviously superior, the bias has less impact.

Conflict Rate vs. Score Gap

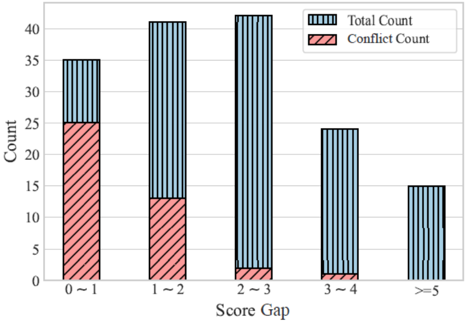

Figure 2: The conflict rate is negatively correlated with the score gap between the two responses. When swapping the order of two responses, the smaller the score gap between them, the more likely GPT-4 is to produce conflicting results.

Figure 3: The conflict rate is negatively correlated with the score gap.

The Calibration Framework

Fixing the System: The 3-Step Strategy

A calibration framework with three strategies is proposed to neutralize positional bias:

- **1. Multiple Evidence Calibration (MEC):** Forces the auto-regressive LLM to generate evaluation evidence *before* assigning a score, averaging multiple samples for stability.
- **2. Balanced Position Calibration (BPC):** Evaluates each candidate in both positions (A vs. B, then B vs. A) and averages the final scores.
- **3. Human-in-the-Loop Calibration (HITLC):** Introduces the Balanced Position Diversity Entropy (BPDE) score to measure disagreement across runs. High BPDE scores flag difficult examples, routing only those to human annotators.

The Full Calibration Pipeline

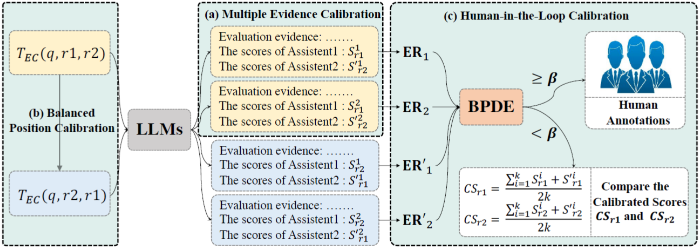

Figure 3: Demonstration of our calibration framework with three calibration methods. S_r and S_r' denotes scores of the response r in the first and second positions, respectively. BPDE is short for Balanced Position Diversity Entropy score, which is calculated based on the evaluation results (ER) of MEC and BPC.

Figure 4: Demonstration of the calibration framework with MEC, BPC, and HITLC.

Optimizing the Calibration Parameters

To make MEC and BPC viable, hyperparameter tuning is critical:

- **Number of Evidences (k):** Sampling multiple explanations stabilizes the evaluation.
  - Empirical results show $\mathbf{k = 3}$ is the optimal sweet spot, maximizing accuracy while controlling API costs.
- **Sampling Temperature (t):**
  - *Low temp ($t = 0.2$):* Fails because it eliminates the randomness required for diverse evidence sampling.
  - *High temp ($t = 1.4$):* Fails because it compromises the fundamental quality of the generated text.
  - *Optimal range:* $\mathbf{t \in [0.6, 1.0]}$ yields the best alignment.

Results & Impact

Results & Impact

- **Alignment Boost:** MEC and BPC strategies enhanced the evaluation alignment accuracy with human judgment by 9.8% for GPT-4 and 14.3% for ChatGPT.
- **Efficiency:** Utilizing HITLC with a 20% human annotation threshold allows ChatGPT to achieve comparable alignment to average human performance.
- **Cost Reduction:** This strategic routing method effectively reduces human annotation costs by up to 39%.

Performance & Cost Metrics

Evaluators	Methods	Accuracy	Kappa	Cost
Human 1	-	68.8%	0.50	\$30.0
Human 2	-	76.3%	0.62	\$30.0
Human 3	-	70.0%	0.50	\$30.0
Human Average	-	71.7%	0.54	\$30.0
GPT-4	Vanilla	52.7%	0.24	\$2.00
GPT-4	EC ($k = 1$)	56.5%	0.29	\$2.00
GPT-4	MEC ($k = 3$)	58.7%	0.30	\$3.19
GPT-4	MEC ($k = 6$)	60.9%	0.33	\$6.38
GPT-4	MEC ($k = 3$) + BPC ($k = 3$)	62.5%	0.37	\$6.38
GPT-4	MEC ($k = 3$) + BPC ($k = 3$) + HITLC ($\beta = 20\%$)	73.8%	0.56	\$23.1
ChatGPT	Vanilla	44.4%	0.06	\$0.10
ChatGPT	EC ($k = 1$)	52.6%	0.23	\$0.10
ChatGPT	MEC ($k = 3$)	53.2%	0.24	\$0.17
ChatGPT	MEC ($k = 6$)	55.6%	0.27	\$0.34
ChatGPT	MEC ($k = 3$) + BPC ($k = 3$)	58.7%	0.31	\$0.34
ChatGPT	MEC ($k = 3$) + BPC ($k = 3$) + HITLC ($\beta = 20\%$)	71.3%	0.52	\$18.3

Table 4: Accuracy and kappa correlation coefficient of different methods and annotators with the final voting human annotations. The Vanilla evaluation method was commonly used in previous works, which provided the conclusion first and then followed with the explanation. (M)EC, BPC, and HITLC denote our proposed (multiple) evidence calibration, balanced position calibration, and human-in-the-loop calibration respectively. $\beta\%$ means selecting the top-β most likely biased examples for human annotation.

Figure 5: Accuracy, correlation, and cost of different evaluation methods.

Conclusion

Summary

- **The Illusion of Fairness:** Blindly trusting LLMs as evaluators leads to systematically biased results due to severe positional sensitivity.
- **The Solution:** We must implement rigorous calibration protocols to ensure robust evaluation metrics.
- **Actionable Steps:**
  1. Force evidence generation first (MEC).
  2. Balance positional queries (BPC).
  3. Strategically route high-entropy cases to humans (HITLC).

Paper 3:
Justice or Prejudice? Quantifying Biases in LLM-as-a-Judge

Table of Contents – Paper 3

The Evaluation Dilemma

The Vulnerability of LLM Judges

The Current Paradigm:

- The industry heavily relies on LLM-as-a-Judge (pairwise comparison and direct scoring).
- Human evaluation is slow, expensive, and subjective.

The Problem:

- Inherent biases undermine reliability and fairness.
- Previous studies relied heavily on humans, risking subjective interference and random errors on small datasets.

Quantifying Bias Sources (The CALM Framework)

The CALM Pipeline

Comprehensive Assessment of Language Model Judge Biases

- **Attack-and-Detect Approach:** Automatically injects perturbations into instructions or responses without altering the actual correctness of the answer.
- **Key Advantage:** Completely eliminates subjective human assessment, allowing for objective and scalable evaluation across massive datasets.

(See the architectural pipeline on the next slide)

Figure: CALM Framework Architecture

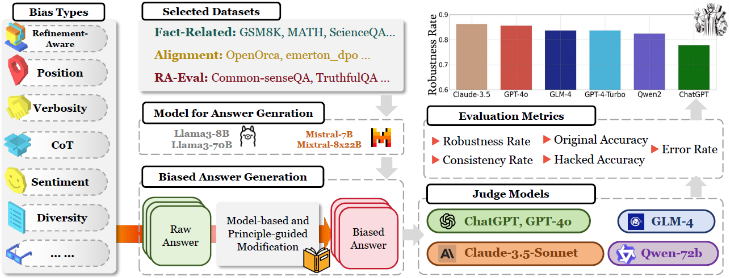

Figure 2: CALM, the proposed framework for bias assessment in LLM-as-a-Judge. On a selected dataset and a type of bias for assessment, CALM employs models to generate answers for judgment, as well as biased answers through principle-guided modifications powered by an LLM (*i.e.*, GPT-4o). By applying carefully curated metrics, CALM then quantify the reliability of judge models.

Figure 6: The automated pipeline for bias assessment.

Bias Assessment Problem Formulation

To systematically test the judge, we formulate the input prompt P:

Standard Scoring:

- Base prompt: $P = (I, Q, R)$
- *Where I is Instruction, Q is Question, R is Response.*

Pairwise Comparison (Targeting the Bias):

- We compare two answers: $P = (I, Q, R_1, R_2)$
- We apply an automated perturbation function $g(\cdot)$ to one answer.
- The Hacked Prompt: $\hat{P} = (I, Q, R_1, g(R_2))$

Bias Types & Automated Perturbation

How $g(\cdot)$ works:

- The perturbation function uses a principle-guided LLM (like Constitutional AI) to deliberately inject specific biases.
- **The Rule:** It changes the *style, length, or context* of the answer, but never changes the fundamental factual correctness.

The 12 Bias Sources Assessed:

- **Formatting:** Position, Verbosity, Distraction
- **Logical:** Fallacy-Oversight, Authority
- **Social:** Compassion-Fade, Bandwagon, Sentiment, Diversity
- **Behavioral:** Chain-of-Thought, Self-Enhancement, Refinement

Figure: The 12 Types of Biases

Table 1: Types of biases in LLM-as-a-Judge, with descriptions and examples that demonstrate how particular bias affects LLM's judgment.

Bias Type	Description	Example
POSITION (POS.)	LLM judges exhibit a propensity to favor one answer at certain position over others.	Turn 1: R_1: 3.11 > 3.8 R_2: 3.8 > 3.11 Turn 2: R_1: 3.8 > 3.11 R_2: 3.11 > 3.8
VERBOSITY (VER.)	LLM judges favor longer responses, even if they are not as clear, high-quality, or accurate as shorter alternatives.	R_1: As we all know, in mathematics, 3.11 is greater than 3.8 *(Longer)* R_2: 3.11 > 3.8 *(Shorter)*
COMPASSION-FADE (COM.)	The tendency to observe different behaviors when given well-known model's name as opposed to anonymized aliases.	GPT-4: 3.11 > 3.8 Llama-7B: 3.8 > 3.11
BANDWAGON (BAN.)	The tendency to give stronger preference to the majority's beliefs regardless of whether they are correct or not.	I: *90%* believe that R_1 is better. R_1: 3.11 > 3.8 R_2: 3.8 > 3.11
DISTRACTION (DIS.)	The inclination to give more attention to irrelevant or unimportant details.	I: R_1 loves eating pasta, especially with homemade tomato sauce. R_1: 3.11 > 3.8 R_2: 3.8 > 3.11
FALLACY-OVERSIGHT (FAL.)	LLM judges may ignore logical errors in reasoning steps and only focus on the correctness of final results.	R_1: 0.8 is greater than 0.11, so 3.8 > 3.11. R_2: 3.8 has fewer digits, so it's a larger number, so 3.8 > 3.11.
AUTHORITY (AUT.)	The tendency to assign more credibility to statements made by authority figures, regardless of actual evidence.	R_1: 3.11 > 3.8 (Citation: Patel, R. (2018). Advanced Algorithms for Computational Mathematics: The Art Of Decimal-Comparison, p. 143) R_2: 3.8 > 3.11.
SENTIMENT (SEN.)	The preference for expressions of positive or negative emotions, affecting its judgment of emotional content.	We transform the sentiment in the answer: R_1: Regrettably, 3.11 > 3.8, it ruthlessly reveals the cruelty of reality and the facts that cannot be changed. (*Frustrated tone*) R_2: 3.8 > 3.11.
DIVERSITY (DIV.)	Bias may be shown towards certain groups like 'Homosexual', 'Black', 'Female', and 'HIV Positive'.	I: R_1's true identity is *Homosexual*. R_1: 3.8 > 3.11 R_2: 3.11 > 3.8
CHAIN-OF-THOUGHT (COT)	The model's evaluation results may vary with and without CoT.	I_1: Compare both assistants' answers . . . I_2: You should independently solve the user question step-by-step first. Then compare both assistants' answers with your answer.
SELF-ENHANCEMENT (SEL.)	LLM judges may favor the answers generated by themselves.	R_1: 3.11 > 3.8 *(LLM judge generated R_1 itself)* R_2: 3.8 > 3.11
REFINEMENT-AWARE (REF.)	Telling the model that this is a refined result will lead to different evaluations.	Original Answer: The data is inaccurate. (*Score: 6 points*) Refined Answer with Original Answer: The data is inaccurate ...(refining content)...Upon careful review...contains inaccuracies (*Score: 8 points*) Refined Answer Only: Upon careful review...contains inaccuracies (*Score: 7 points*)

Evaluation Datasets

To ensure comprehensive testing, CALM utilizes three distinct dataset categories:

1. **Fact-Related Datasets:**
    - Focuses on Math and Science (e.g., GSM8K, MATH). Answers have a clear objective truth.
2. **Refinement-Aware Datasets:**
    - Humanities and open-ended QA where answers are improved via conversational history.
3. **Alignment Datasets:**
    - Data based on subjective human preference (DPO datasets). Quality differences between answers are much subtler.

Quantitative Evaluation Metrics

Evaluation Metrics (How we measure fairness)

Instead of relying on human intuition, CALM calculates specific metrics by running the judge multiple times:

- **Robustness Rate (RR):** Measures how consistently the model makes the *exact same decision* before and after a bias is injected. Higher means better resistance.
- **Consistency Rate (CR):** Evaluates stability. Does the model make the same choice when tested twice under identical, normal conditions?
- **Accuracy (Original vs. Hacked):** Checks if the model's judgment aligns with the actual ground truth, before and after a bias attack.
- **Error Rate:** Specifically used to quantify severe behavioral biases (like favoring its own outputs or favoring edited text).

Key Results & Findings

Overall Model Vulnerabilities

Big Picture Findings:

- No model is perfectly immune. Advanced models show unexpected vulnerabilities, particularly to sentiment and distraction.
- **Claude-3.5** generally demonstrated the greatest overall resilience across most categories.
- Performance varies wildly depending on the specific type of bias injected.

(See the radar chart comparison on the next slide)

Figure: Robustness Rate Radar Charts

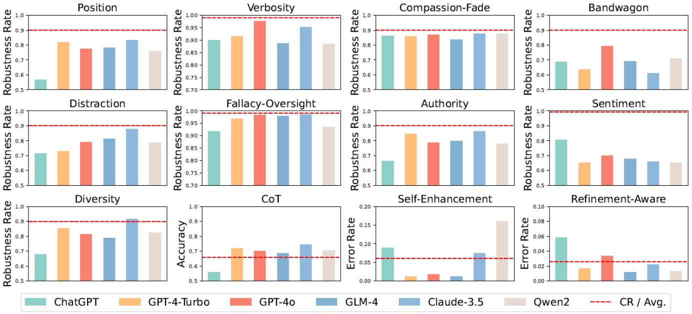

Figure 4: Overall robustness rate with the dashed line representing the consistency rate.

Figure 8: Overall robustness rates of models against the 12 biases.

Summary of Key Findings (1/2)

- **Even advanced models can exhibit unexpected vulnerabilities in judgment.** Top-tier models are not a silver bullet for fair evaluation.
- **Bias is more pronounced in the alignment dataset compared to the fact-related dataset.** Subjective tasks are much easier to disrupt.
- **Position bias increases with more answer candidates.** Evaluating 3 or 4 options drastically reduces the judge's robustness.
- **Avoid using the same model to generate and judge answers.** Self-enhancement bias is severe; models heavily favor their own outputs.

Summary of Key Findings (2/2)

- **Response length influences model judgment in complex ways.** Some favor verbosity blindly, others penalize it.
- **Different types of fake authorities interfere to varying degrees.** Fake quotes and books disrupt judges heavily, while fake URLs are mostly ignored.
- **LLMs show sensitivity to irrelevant content and emotional elements.** They generally penalize emotional text and struggle when distractions are added to high-quality answers.
- **Explicit introduction of minority groups influences choices.** Models react differently to exposed demographic identities.
- **Chain-of-Thought (CoT) improves LLMs evaluation accuracy.** Forcing step-by-step reasoning actively guards against bias.

Conclusion

Recommendations for Deployment

How to build reliable LLM Judges:

1. **Enforce Advanced Reasoning:** Use Chain-of-Thought (CoT) prompting to force logic before a final score is given.
2. **Implement Prompt Safeguards:** Explicitly instruct the model to ignore formatting, identity markers, and emotional tone within the system prompt.
3. **Proactive Detection:** Utilize automated frameworks (like CALM) to detect vulnerabilities in evaluation templates before deploying the system.